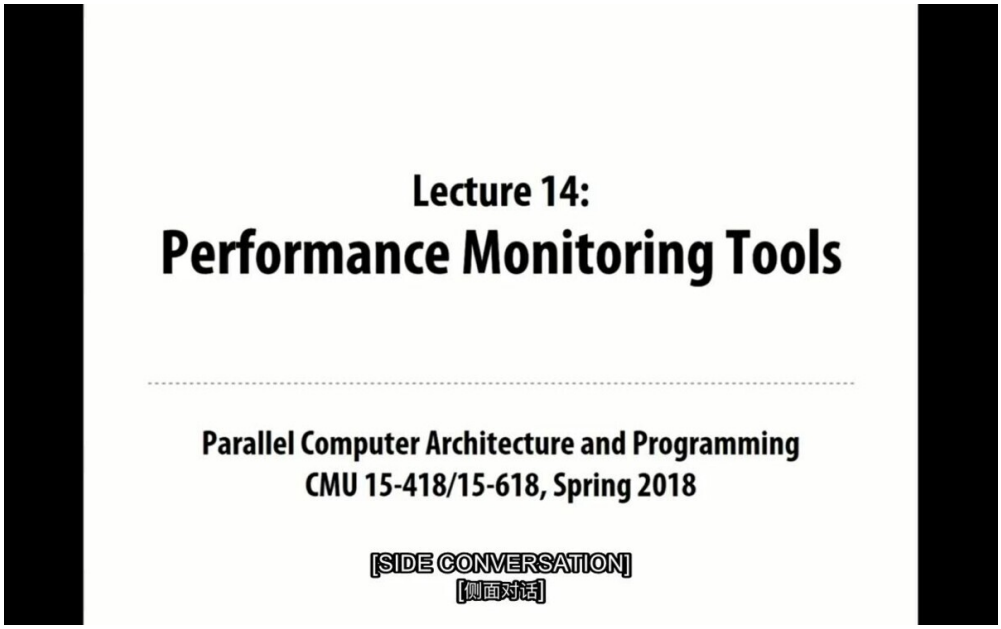
课程笔记

性能测量与调优：由外向内定位瓶颈

CMU 15-418/618 并行计算机架构与编程 · Lecture 18（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（Kayvon Fatahalian 讲授，B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 16 分 42 秒
视频链接：`https://www.bilibili.com/video/BV1SX4y1W7fG/`

目录

1 “我的程序很慢”：先弄清楚是哪一种慢 2

1.1 测量比猜测有价值 . . . 2
1.2 本讲的例子来自具体学期 . . . 3
1.3 本章小结 . . . 3

2 第一层：机器上还有谁在跑 3

2.1 top . . . 3
2.2 那个让人困惑的百分比 . . . 4
2.3 一定要用 top 吗 . . . 5
2.4 本章小结 . . . 5

3 第二层：函数级计时 5

3.1 插桩的基本问题 . . . 5
3.2 重访 Amdahl 定律 . . . 7
3.3 gprof . . . 7
3.4 本章小结 . . . 8

4 第三层：硬件性能计数器 8

4.1 架构提供的计数器 . . . 8
4.2 perf stat . . . 9
4.3 分时复用 . . . 10
4.4 读懂默认输出 . . . 10
4.5 本章小结 . . . 11

5 第四层：采样分析定位到行 12

5.1 perf record 与 report . . . 12
5.2 一个必须理解的精度限制 . . . 12
5.3 从函数到行 . . . 13
5.4 本章小结 . . . 13

6 案例：一个指针间接寻址的代价 14

6.1 先看反汇编 . . . 14
6.2 真正的瓶颈不在比较，而在上一行 . . . 15
6.3 改动与效果 . . . 15
6.4 本章小结 . . . 15

7 工具的代价与使用边界 16

7.1 插桩带来的开销 . . . 16
7.2 本章小结 . . . 16

8 总结与延伸 16

1 “我的程序很慢”：先弄清楚是哪一种慢

这堂由 Kayvon Fatahalian 讲授的课是一个补充专题。他开场描述了一个他经常遇到的场景：学生走进办公室说，我的代码很慢、好像用了很多内存、被 SIGKILL 杀掉了。

他的第一句话是：有些情况下答案是去用调试器——但那只在 SIGKILL 这类情形下偶尔有效，而 SIGKILL 往往正是因为内存用得太多。

1.1 测量比猜测有价值

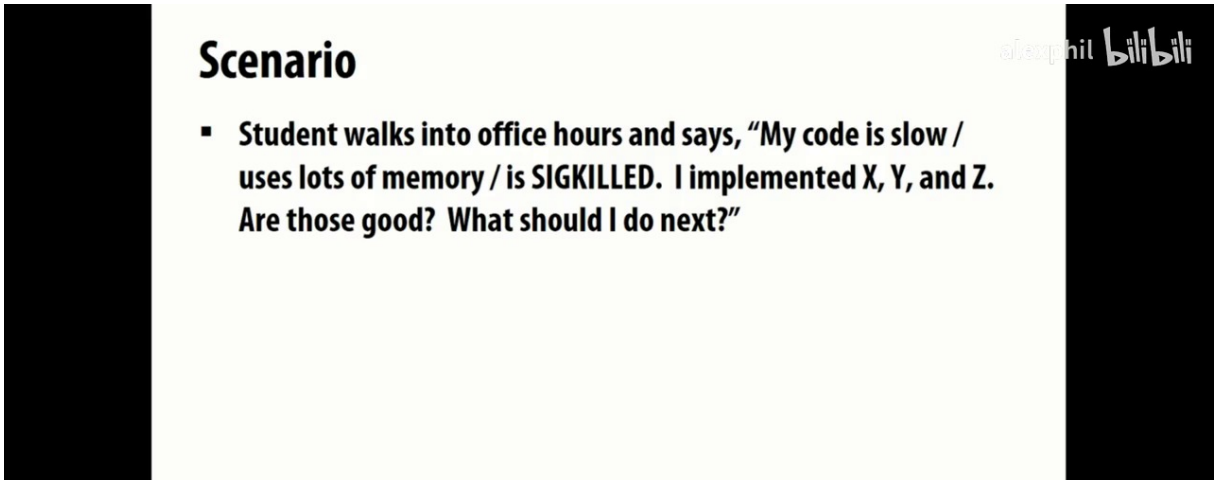


图 1: 场景：学生走进办公室说” 我的代码很慢、用了很多内存、被 SIGKILL 了”[1]

讲者给出本讲的核心命题：



图 2: 我的程序在做什么：测量比直觉更有价值[2]

[1] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

”慢”是一个症状，不是诊断

程序慢可能意味着：机器被别人的进程占满、内存不足导致换页、某个函数占了七成时间、缓存缺失率过高、或者某处数据布局导致每次访问都多走一级指针。这五种情况的解法完全不同，而仅凭”慢”这一个观察无法区分。工具的作用是把症状转化为可定位的指标。

1.2 本讲的例子来自具体学期

讲者说明他的幻灯片基于 2016 年春季学期的作业三，当时是图处理负载：图中有数百万到数千万个节点，代码跑在 GHC 机器与 Xeon Phi 上。他坦承现在这门课的作业三换成了鼠群模拟，因此部分幻灯片只能说”当年是这样”。

他也提到自己 2000 年秋季就在这个教室上过课，那是他来 CMU 的第一堂课。这段插曲之后他转入正题。

1.3 本章小结

”我的程序慢”是一个症状而非诊断，可能对应机器被占用、内存不足、函数级热点、缓存缺失、数据布局等多种原因。工具的价值在于把症状转成可测量的指标。本讲的例子来自 2016 年春季的图处理作业。

下一章从最外层开始：先确认机器上还有没有别人。

2 第一层：机器上还有谁在跑

2.1 top

讲者给出的第一个工具就是 top。他描述了一次真实的困惑：SSH 进一台机器，看到有 50 个用户在线，于是怀疑是不是同学们都在跑代码。但 50 个人都登录同一台机器、且都恰好空闲地盯着屏幕，这不太可能——所以大概是有人在跑。

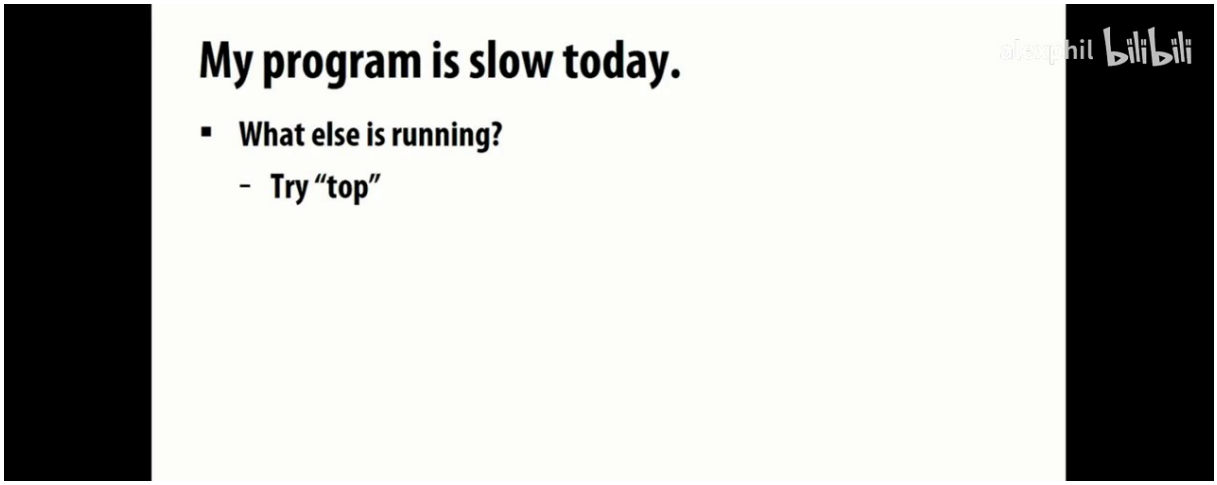


图 3：机器上还有什么在跑：先用 top 看整体负载[3]

[3]视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

结果那一次并不是。top 显示 99.9% 空闲，这台机器基本闲置。于是可以判断：慢的原因确实在自己的代码里。

top 还能看内存：机器共约 16 GB，空闲约 8 GB。

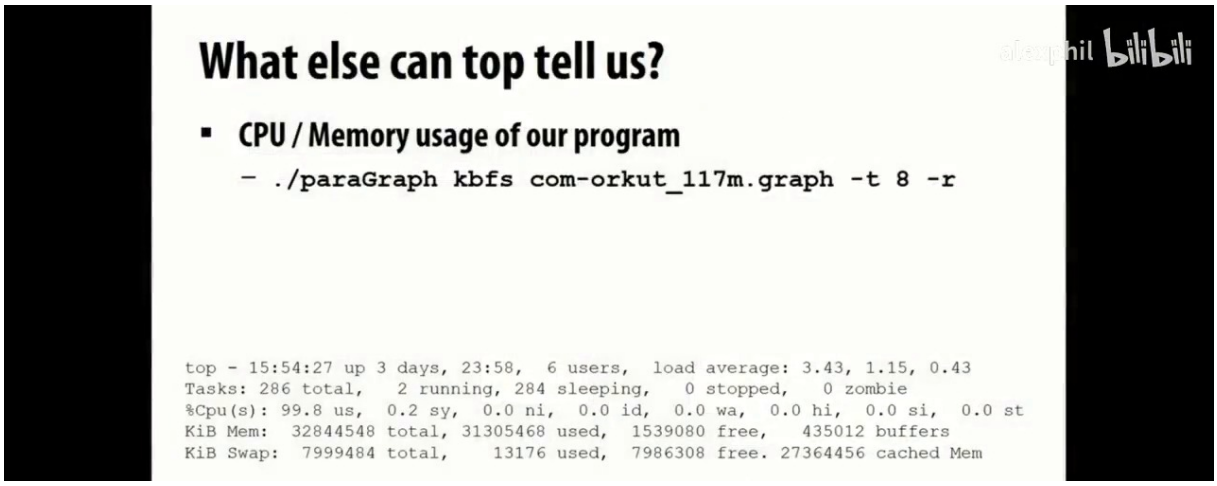


图 4: top 还能告诉我们什么：程序的 CPU 与内存使用[4]

2.2 那个让人困惑的百分比

讲者在这里专门停下来讲一个陷阱。他跑图处理负载并指定八个线程，top 显示他用了接近 800% 的 CPU。

讲者还提到另一次运行使用了 600% 的 CPU，那一次耗时 33 秒。这类数字经常引起争论。他提到关于” 百分比怎么加” 存在长期争论：这台机器有八个硬件线程，当代码用满八个线程时，你用的是系统的 100%，还是 800%？连工具本身都不一致——同一屏上系统整体 CPU 使用率显示 99%，而他的程序显示 796%。

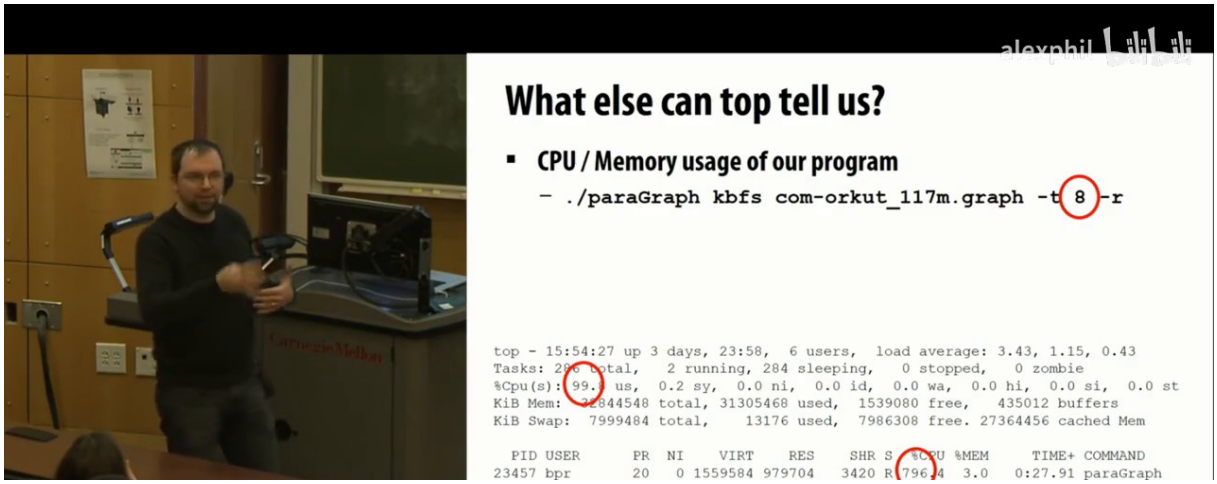


图 5: top 的 CPU 百分比口径：按逻辑线程累加，八线程满载即 800%[5]

讲者的解释很清楚：这个数字告诉我们” 聚合来看有 99% 的 CPU 在被使用”，而恰好是他一个人用掉了相当于八个 CPU 的负载。

[4] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

[5] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

$$\mathrm{CPU\%}_{\mathrm{program}} \approx N_{\mathrm{threads}} \times 100\%$$

- N_{threads}：该程序实际使用的硬件线程数，本例为 8；
- 因此显示 796% 意味着接近八核满载；
- 系统整体百分比则是所有进程的聚合，上限是 100%。

读 top 前必须先确认百分比口径

不同工具、不同版本对 CPU 百分比的定义并不统一：有的按单核为 100%，有的按整机为 100%。看到 200% 时先问自己” 这台机器有几个硬件线程”，否则会把满载误读为异常，或把 25% 的利用误读为充分利用。

2.3 一定要用 top 吗

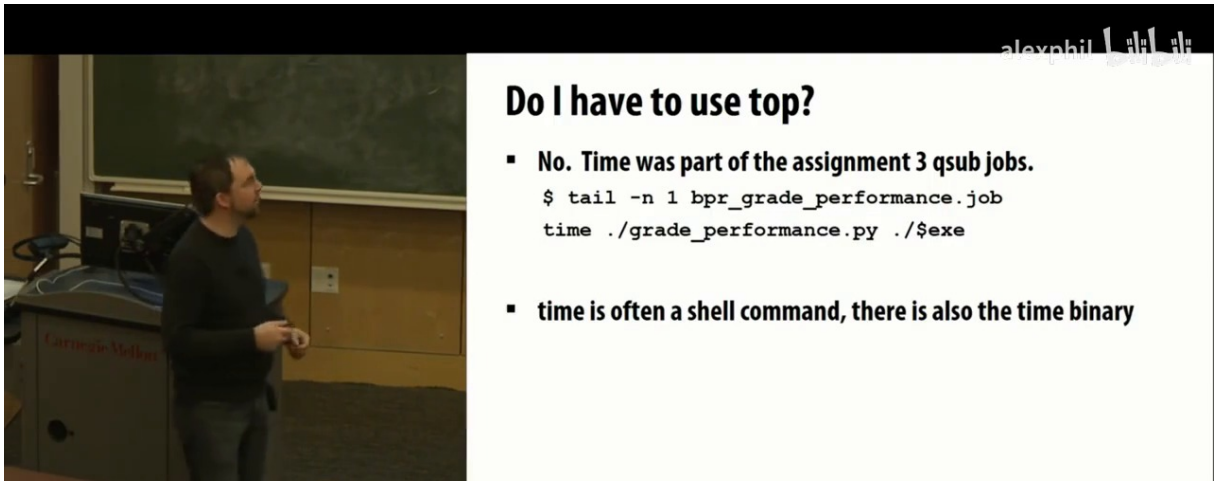


图 6: 一定要用 top 吗：作业三的计时也是评分的一部分[6]

讲者的回答是不一定。作业三本身就包含计时，提交时会跑并给出性能评分。因此也可以直接用评测脚本的输出，而不必手工盯着 top。

2.4 本章小结

第一层是系统级：用 top 确认机器是否被他人占满、内存是否充足。这里有一个必须警惕的细节——CPU 百分比的口径按逻辑线程累加，八线程满载会显示 800% 而非 100%。

确认了机器没问题，就可以转向” 我的代码里时间花在哪”。

3 第二层：函数级计时

3.1 插桩的基本问题

讲者先给出一个概念框架：插桩的核心问题是” 在什么时候注入测量”。可以在编译时注入，也可以在运行时注入。

[6] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

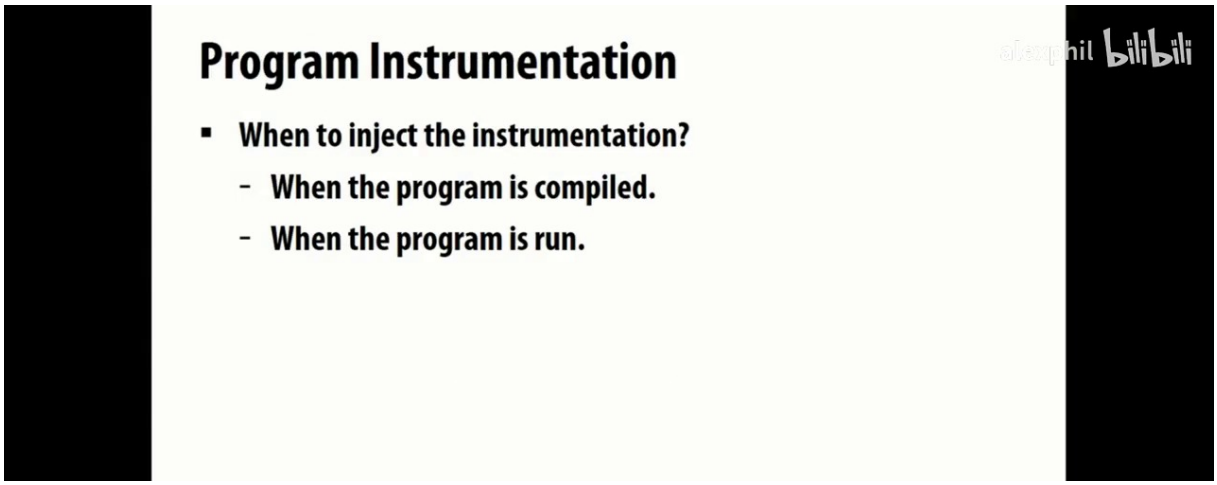


图 7: 程序插桩：在编译时或运行时注入测量代码[7]

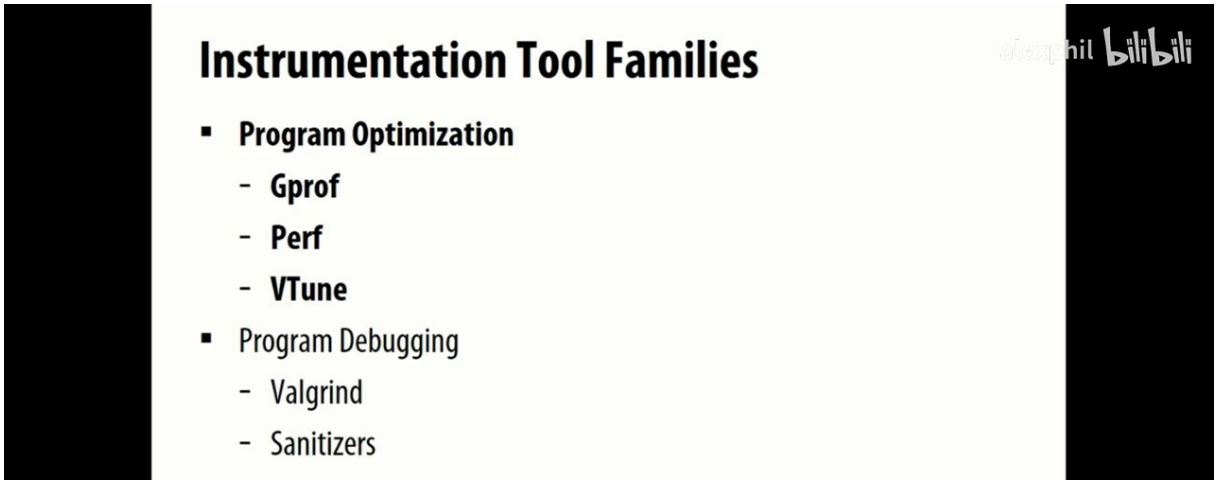


图 8: 插桩工具的几大类别：性能优化类与调试类[8]

两类工具回答不同问题

性能优化类工具回答” 时间花在哪里”，它假设程序行为正确、只是不够快。调试类工具回答” 程序为什么行为不对”，它关注状态与流程。用错类别会得到大量无关信息：拿调试器去找性能瓶颈，或拿 profiler 去查逻辑错误。

他把工具分成两大类：性能优化类（gprof、perf、VTune 等）与程序调试类。

[7] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。
[8] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

3.2 重访 Amdahl 定律

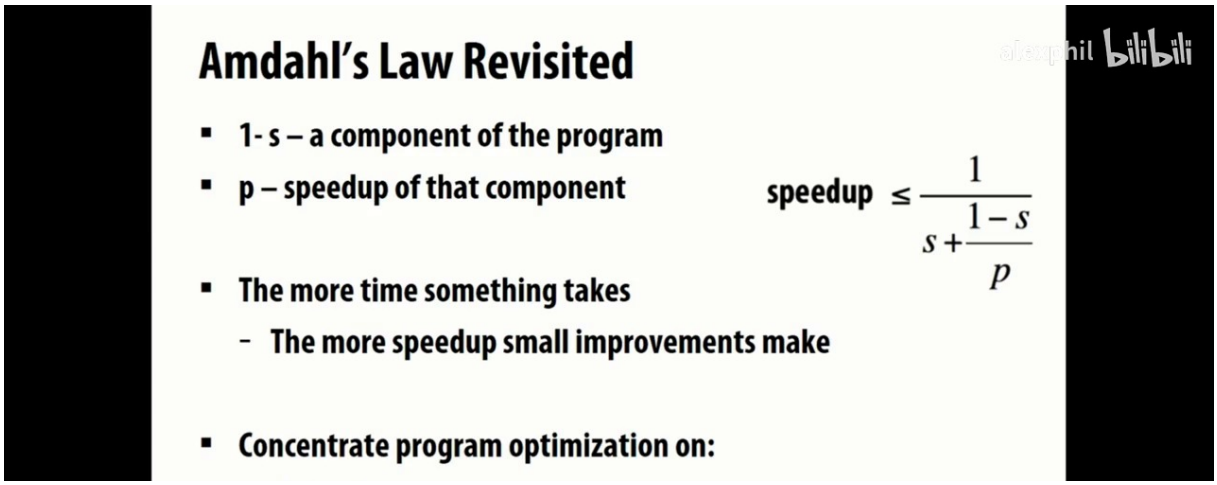


图 9: 重访 Amdahl 定律：优化占比为 s 的部分，整体加速受其限制[9]

讲者在这里用 Amdahl 定律说明为什么必须先定位热点：

$$S_{\text{total}} = \frac{1}{(1-s) + \dfrac{s}{p}}$$

- s：被优化部分占程序总时间的比例；
- p：该部分自身获得的加速倍数；
- 当 $p \to \infty$ 时，S_{total} 上界为 $1/(1-s)$。

若某函数只占 5% 时间，即使把它优化到零，整体也只快 5%。而若它占 50%，同样的努力收益大十倍。

3.3 gprof

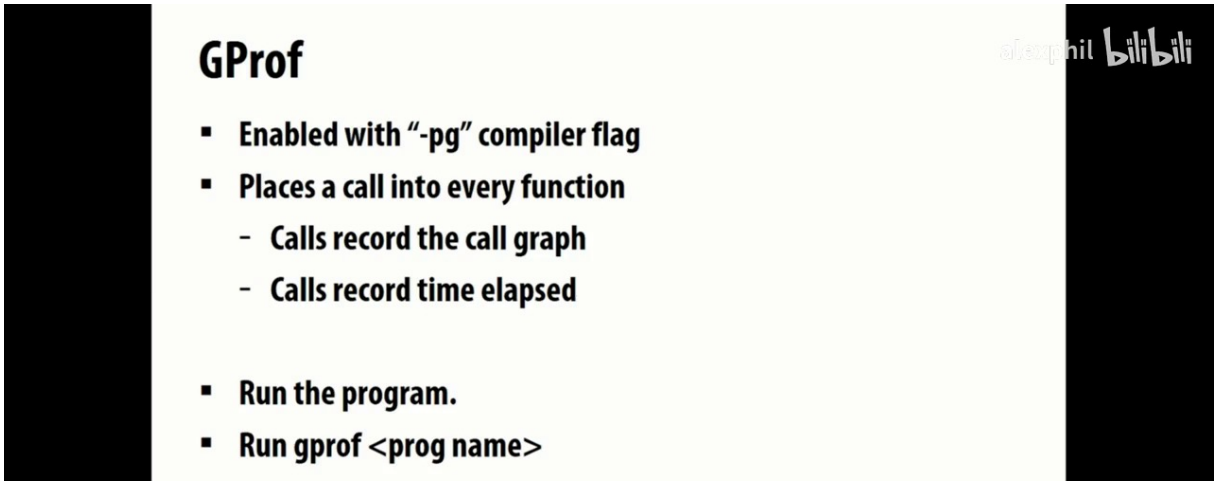


图 10: gprof：用 -pg 编译选项启用，在每个函数处插入调用[10]

[9] 视频画面时间区间：00:16:20–00:16:35。若为裁剪图，时间区间仍对应原视频画面。

gprof 通过 `-pg` 编译选项启用，它在每个函数处插入调用，记录调用关系与耗时。

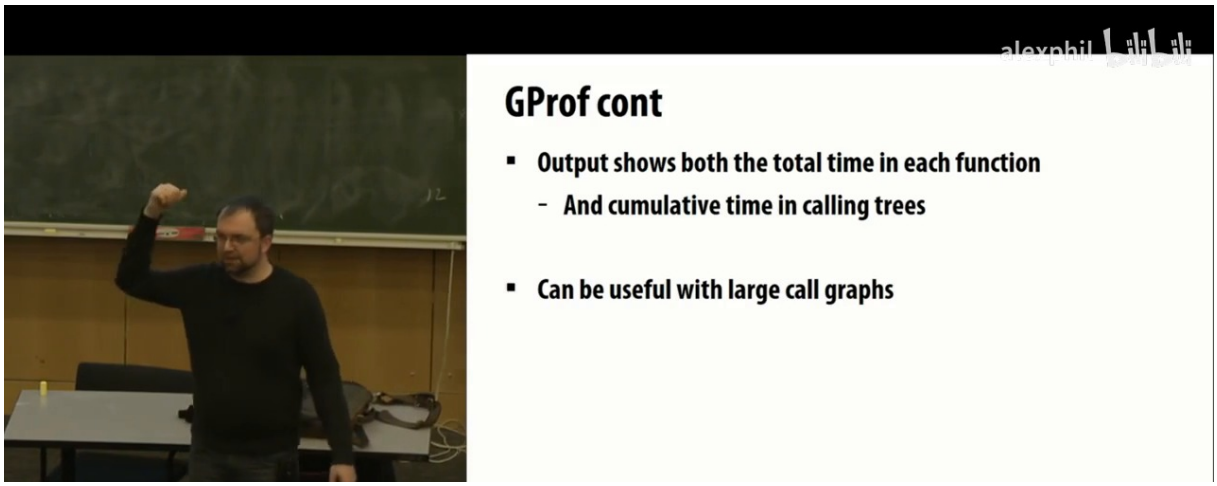


图 11: gprof 输出：各函数总耗时、调用树累积时间与调用次数[11]

输出会同时给出每个函数的总耗时、调用树中的累积时间，以及调用次数。

讲者给出实测结果：70% 的时间花在 `build_incoming_edges`（构建整张图），30% 花在 `pagerank` 上，还显示了调用次数——有的是 18 次，有的是 160 万次。

要排除与评分无关的框架代码

`build_incoming_edges` 是教师提供的代码，学生无法改变它，而且计时只针对 pagerank 部分。因此虽然它占了七成时间，却可以整个排除。这一步判断不是工具给出的，而是人根据评分规则做出的。

3.4 本章小结

第二层是函数级：gprof 用 `-pg` 插桩，给出每个函数的时间占比与调用次数。Amdahl 定律说明为什么必须先做这一步——占比决定收益上限。但工具给出的占比必须结合评分规则解读，否则会把无关的框架代码当成优化目标。

4 第三层：硬件性能计数器

4.1 架构提供的计数器

讲者转向硬件层面。现代架构暴露了一组性能计数器，可以统计缓存缺失、分支预测失败、每周期指令数等。

[10] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[11] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

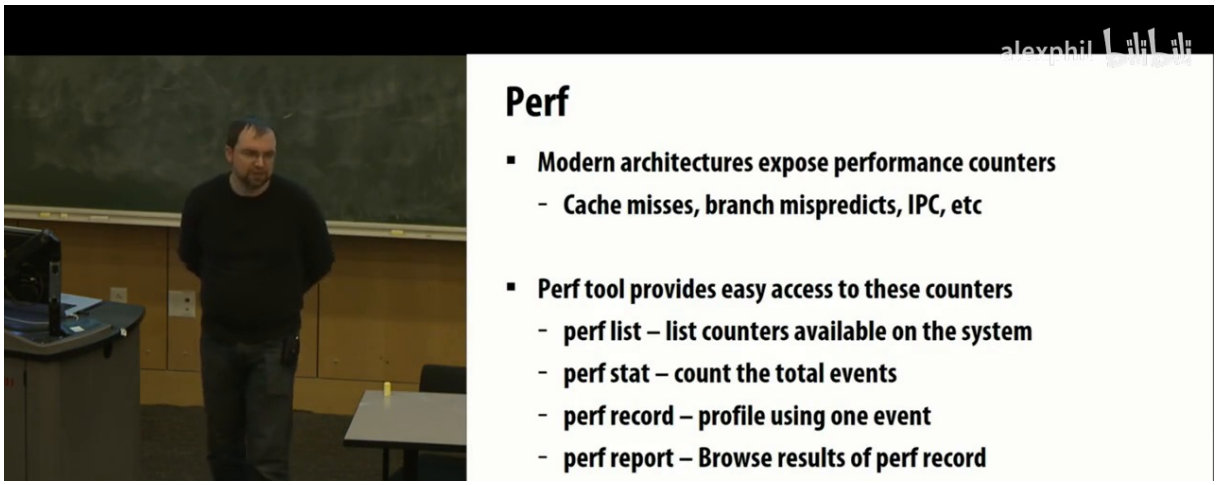


图 12: perf：访问现代架构暴露的性能计数器[12]

这些计数器是特权资源，因此相应工具需要由 root 安装。如果机器上没有，要么自己成为 root，要么找人装。

4.2 perf stat

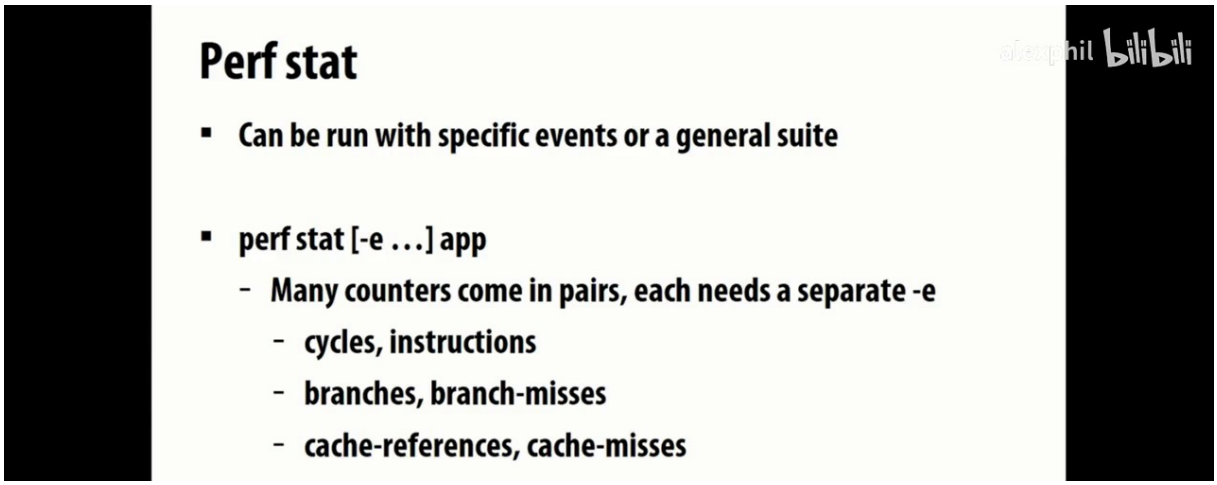


图 13: perf stat：可指定事件，也可用默认集合[13]

`perf stat` 是最简单的入口：可以指定事件，也可以让它收集一组常用计数器。

讲者强调了一条极易被忽视的规则：** 计数器必须成对使用 **。

比如有一个计数器统计分支总数，另一个统计分支预测失败数。如果只告诉你” 预测失败了 100 万次”，你无法判断这是 90% 的失败率还是 1%。缓存同理——” 有 100 万次缓存缺失” 是好是坏，取决于总访问数。

[12] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

绝对计数必须除以总数才有意义

任何计数器给出的都是绝对量。缺失数、预测失败数、周期数本身都无法判断好坏，必须与对应的总数相比构成比率。因此收集计数器时要有意识地成对选取，否则拿到的是一堆无法解读的数字。

4.3 分时复用

分时复用会掩盖阶段性行为

程序若明显分为” 构图” 与” 计算” 两个阶段，分时复用会把两个阶段的数据混合外推，得到的比率既不代表构图也不代表计算。此时应当把两个阶段拆成两次运行分别采集，或改用基于采样的 perf record。

若一次执行中选了超过四个计数器，硬件会做分时复用：周期性关掉一个、打开另一个，假设你的代码行为足够稳定，然后把各段结果聚合起来，当作全程都在采集。

分时复用的前提是代码行为稳定

硬件能同时采集的计数器数量有限（通常四个）。超过时它轮流采样并外推。若程序有明显的阶段性（例如先构图后计算），这种外推会把两个阶段的数据混在一起，得到误导性的比率。此时应分段运行，或用采样方式替代。

4.4 读懂默认输出

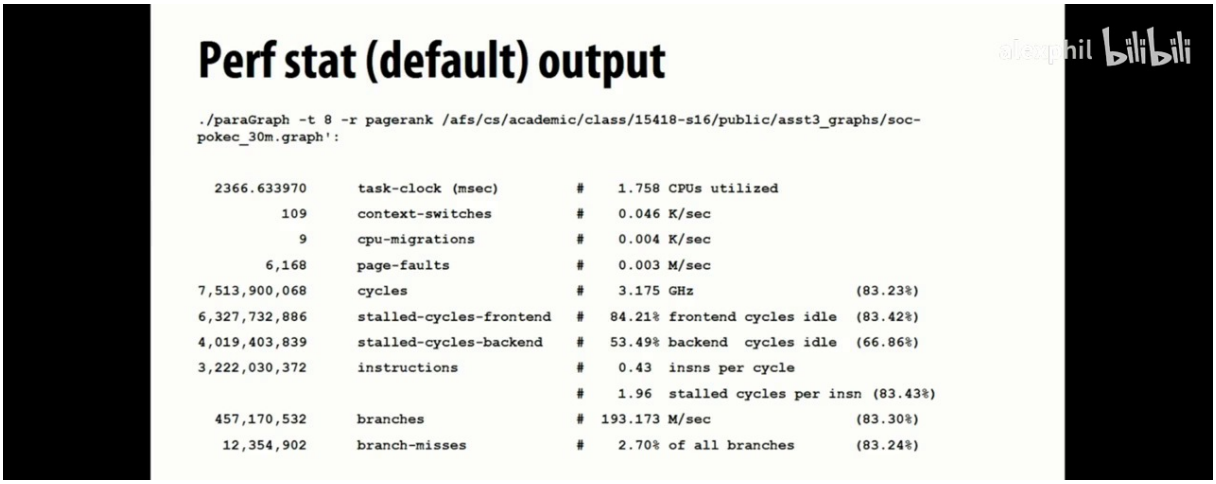


图 14: perf stat 默认输出：耗时、周期、停顿、指令数与 IPC[14]

讲者展示了 page rank 的实测输出，其中几个数字很关键：

耗时约若干毫秒、使用了若干 CPU；总周期数；前端与处理器内部的停顿时间；执行的指令数。由此算出每周期只执行 0.4 条指令——讲者的评语是” 这不太好”。

分支方面：只有 2.7% 的分支被预测错误，这是正常的。

于是问题是：瓶颈在哪？这个默认集合里缺了什么？

[14] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

答案很自然：缺缓存。讲者于是加上缓存计数器：

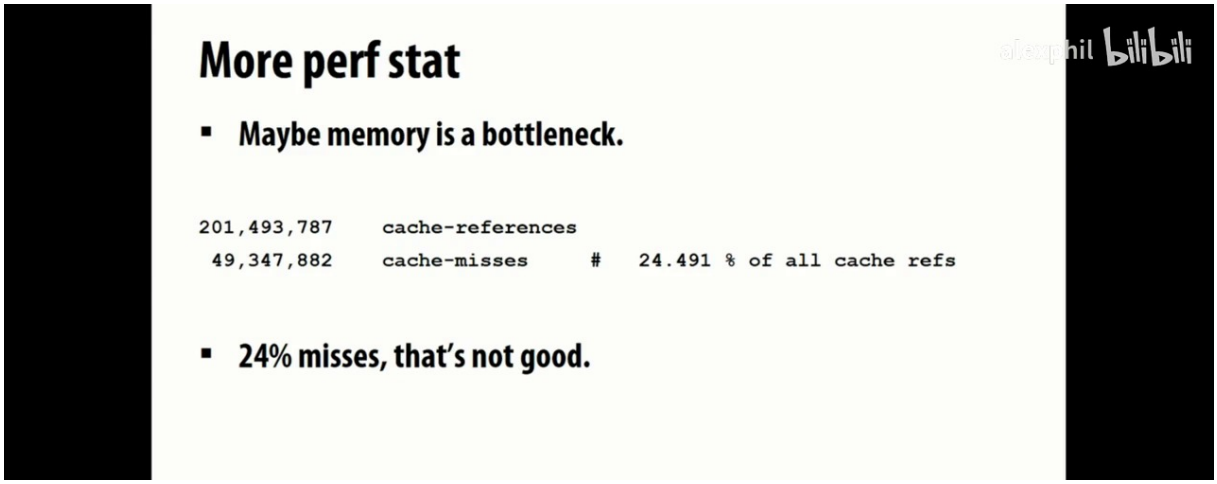


图 15: 更多 perf stat：缓存引用数与缺失数[15]

结果是约 5000 万次缓存缺失，而总访存约 2 亿次，缺失率 24.5%。讲者的评语同样是” 这不太好”。

缺失率由两个计数器相除得到：

$$\text{miss rate} = \frac{N_{\text{miss}}}{N_{\text{ref}}} = \frac{49{,}347{,}882}{201{,}493{,}787} \approx 24.5\%$$

- N_{miss}：`cache-misses` 计数器，约 4935 万；
- N_{ref}：`cache-references` 计数器，约 2.01 亿；
- 两者必须成对采集，单独任何一个都无法判断好坏。

用排除法读性能计数器

讲者的推理路径值得记住：先看 IPC（0.4 偏低），再看分支预测（2.7% 正常，排除），于是怀疑内存，加上缓存计数器确认缺失率 24.5%。每一步都是用一个指标排除一种可能，而不是一次性把所有计数器看一遍。这套顺序能把” 哪里不对” 缩小到具体子系统。

4.5 本章小结

第三层是硬件计数器。perf stat 给出周期、指令、IPC、分支与缓存等指标，但绝对数必须配总数才有意义，超过四个计数器会分时复用。读的方法是用排除法：IPC 偏低 → 分支正常 → 怀疑内存 → 缓存缺失率确认。

但函数级与子系统级的信息还不够，还需要知道具体是哪一行代码。

[15] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

5 第四层：采样分析定位到行

5.1 perf record 与 report

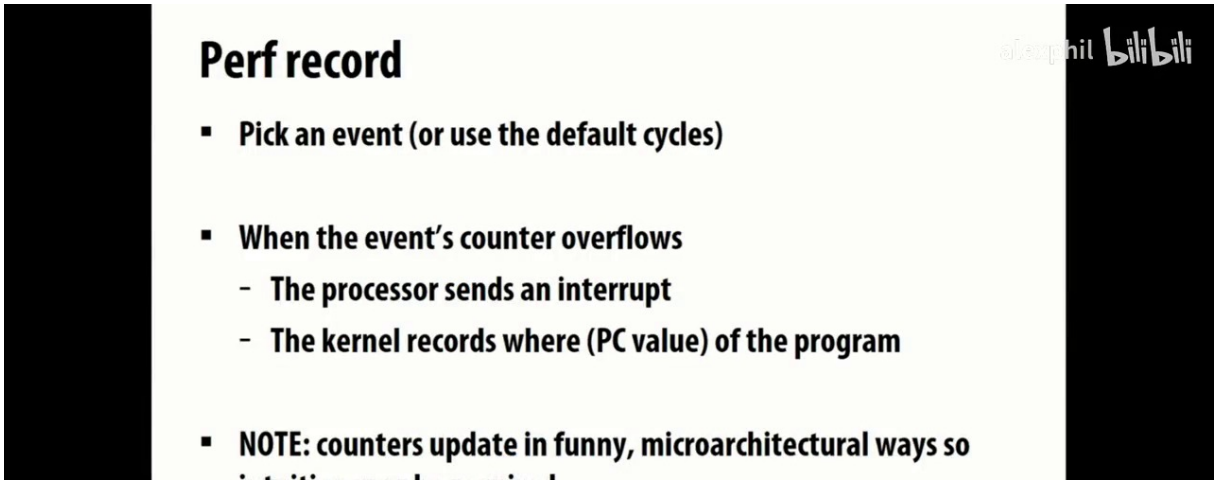


图 16: perf record：选择事件，计数器溢出时中断并记录位置[16]

`perf record` 选择一个事件（默认是周期），当该事件的计数器溢出时触发中断，记录当时执行到哪里。

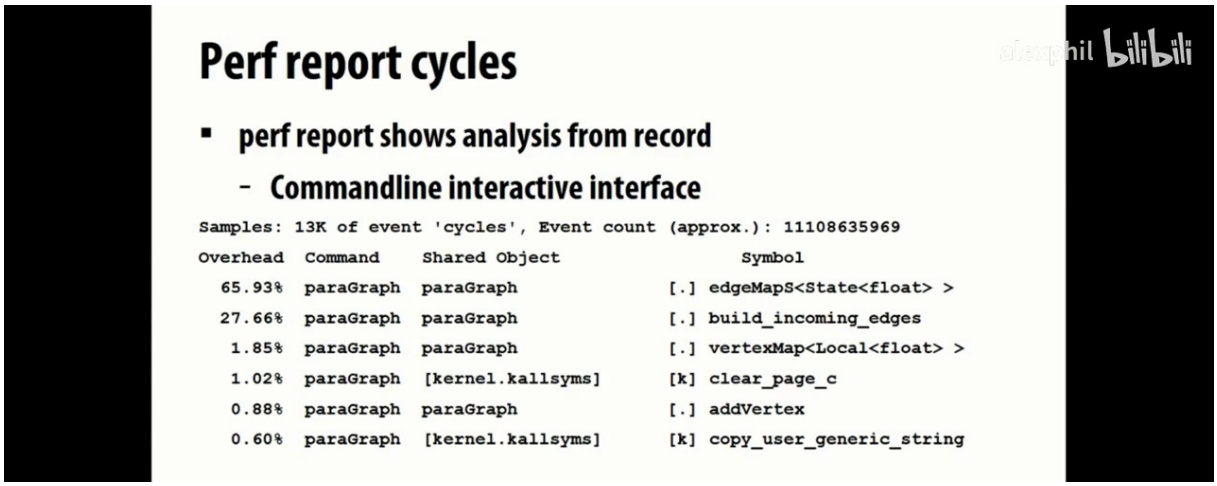


图 17: perf report：从记录生成交互式分析[17]

`perf report` 把记录转成分析。讲者说明采样点会自然集中在真正发生事件的位置——有 5000 万次缺失，若每 5 万次中断一次，就得到约 1000 个采样点，它们会散布在真正发生缺失的地方。

5.2 一个必须理解的精度限制

讲者在这里引用了架构手册的一段话，值得完整记录：由于微架构中存在延迟，从事件产生到溢出中断之间有时间差，因此很难在紧邻事件的位置产生中断。

这意味着工具告诉你的位置，未必就是事件真正发生的那条指令。

[16] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

采样位置可能偏移一到两条指令

看到”某条不访问内存的代码行上有数百万次缓存缺失”时，不要断定工具坏了。真正的事件通常发生在其前后一到两条指令处。这要求使用者带着判断去复核周围的代码，而不是机械相信工具指向的那一行。

讲者说他的经验是偏移通常只有一两条指令。而能把范围缩小到两三条汇编指令，已经比之前所有工具都好得多。

5.3 从函数到行

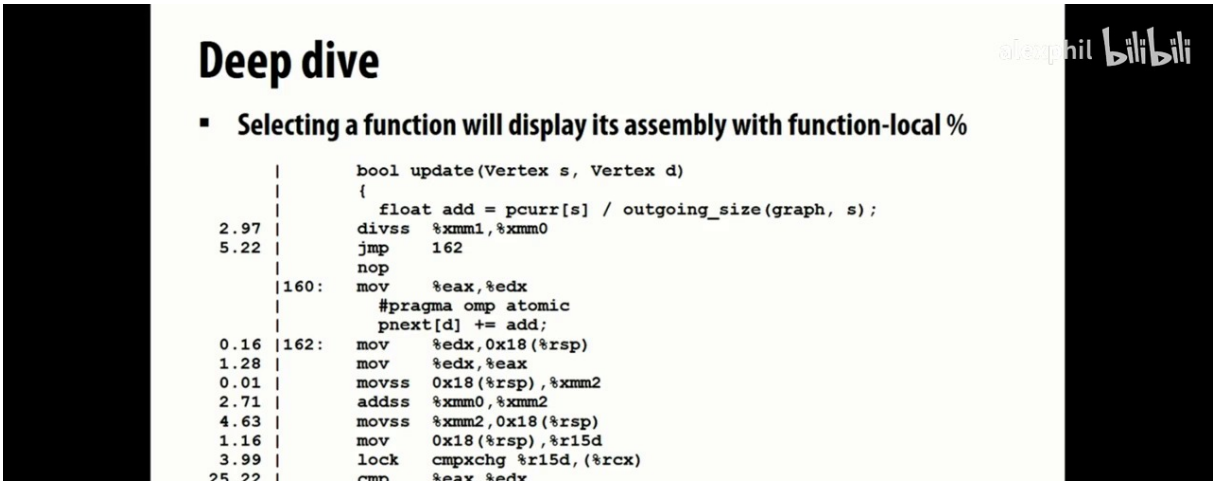


图 18: 深入：选中函数会显示其汇编与函数内的热点分布[18]

有了采样结果，就能看到缓存缺失在各函数间的分布。`edge_map` 这个自写例程占了 47 个百分点。构图代码占了另外 46 个百分点。两者合计 93 个百分点，几乎覆盖全部缺失。这已经很有用，但还可以继续深入——选中某个函数，工具会显示它的汇编，并标出函数内的热点百分比。

5.4 本章小结

第四层是采样分析：perf record 在事件溢出时中断记录位置，perf report 汇总成函数级与行级的热点。关键限制是微架构延迟会让位置偏移一到两条指令，因此必须结合人的判断复核。

这一层已经能把问题定位到具体代码行。最后一章看一个真实的优化案例。

[18] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

6 案例：一个指针间接寻址的代价

6.1 先看反汇编



图 19: 反汇编：究竟是什么在消耗 KBFS 的时间[19]

讲者用 KBFS 负载（在图上做多次广度优先搜索）说明下一步该做什么。这一次，超过 80% 的执行时间都在他自己写的代码里——不能再归咎于教师提供的构图代码。

热点行常常紧邻真正的元凶

采样工具指向的行未必就是问题所在。本例中报出的位置是一个比较操作，而真正的开销来自它上方那一行的间接寻址。读报告时应把热点行连同前后几条指令一起看，而不是只盯被标记的那一行。

于是他看那个大循环：里面有一个操作，检查源与目标是否都访问过；若没有就做事，若有就跳过。

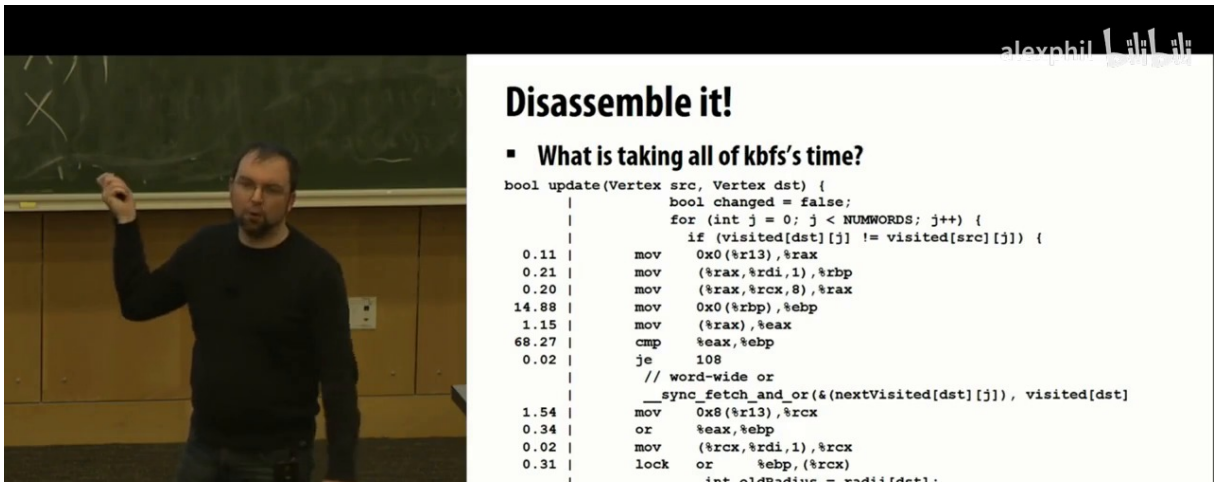


图 20: 双重指针数组造成两级间接寻址，每次访问都多付一次跳转[20]

[19] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

[20] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

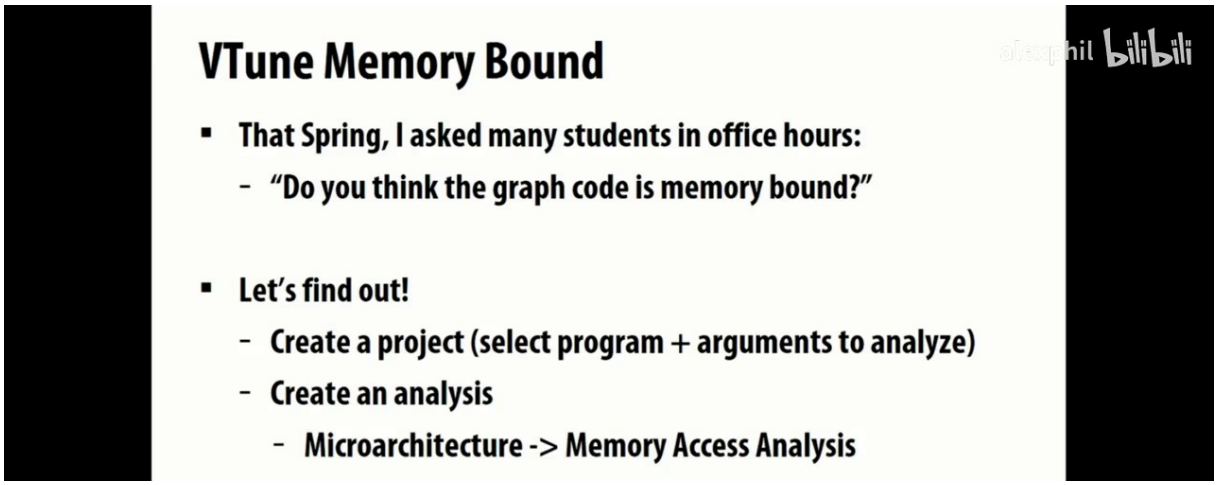


图 21: VTune 的 Memory Bound 视图：这类问题属于内存受限[21]

6.2 真正的瓶颈不在比较，而在上一行

讲者指出时间并不花在那个比较上，而是花在它上方那一行内存访问。

关键在于 `visited` 的类型：它被实现成指向指针数组的指针，看起来像二维数组，实际每次访问都要经过两级间接寻址。这个代价很大。

数据布局的代价常被算法分析忽略

从算法复杂度看，二维数组的访问是 $O(1)$；但从硬件看，" 指针数组 + 每行单独分配" 意味着每次访问多一次内存跳转，且各行的物理位置不连续，缓存局部性也差。复杂度相同的两种实现，实测可以差近十倍。这类问题只能通过工具定位，无法从算法推导出来。

6.3 改动与效果

讲者的改动很简单：去掉一级间接寻址。结果代码跑得显著变快，实测减速倍数为 9.8 倍——也就是说原来那一级指针让代码慢了近十倍。

他坦率地说当时班里没人找到这个原因——因为那时他还没讲过这堂课，大家都没有这套方法。这正是本讲存在的理由。他补充说，后来他用 VTune 的 Memory Bound 视图重新看这个问题，工具会直接把它归类为" 内存受限"，这让定位变得容易得多。这也说明不同工具的抽象层次不同：perf 给的是原始计数器与采样点，VTune 这类工具会在其上做一层归类，把纷杂的数字压缩成" 你的程序受限于内存" 这样一句结论。两者各有价值：原始数据更可信，归类结论更易读。他还提到曾问过很多同学" 你觉得这段代码受限于什么"，多数人的直觉判断与实际测量不符——这也正是为什么必须测量而不能凭感觉。

6.4 本章小结

KBFS 案例说明真正的瓶颈常在数据布局：双重指针数组让每次访问多付一级间接寻址，去掉后减速倍数从 9.8 倍回到正常。这类问题在算法复杂度上完全不可见，只能靠工具定位。它也印证了采样报告需要连同前后指令一起读——报出的行往往不是真正的元凶。

工具本身有代价，这决定了它给出的数字该怎么用。

[21] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

7　工具的代价与使用边界

7.1　插桩带来的开销

讲者也给出这类工具的代价：插桩会带来数倍到数十倍的开销（他提到的量级是从 4× 到 20×），而且开销本身会改变程序的时间分布。

因此插桩测得的耗时需要按开销倍数修正：

$$T_{\text{real}} \approx \frac{T_{\text{instrumented}}}{1+o}$$

- $T_{\text{instrumented}}$：带插桩运行时测得的耗时；
- o：插桩带来的额外开销比例，量级从 4 到 20；
- 该修正只在开销均匀分布时有效，否则只能用于比较相对占比。

插桩开销会扭曲被测量的对象

gprof 在每个函数入口插桩，对频繁调用的小函数影响尤其大；缓存模拟器需要逐条模拟访存，比真实硬件慢几个数量级。因此用插桩工具得到的是”相对分布”而不是”绝对耗时”。要测真实性能，应当用未插桩的版本。

7.2　本章小结

KBFS 案例说明真正的瓶颈常在数据布局而非算法：双重指针数组让每次访问多付一级间接寻址，去掉后显著变快，这个倍数达到近十倍。插桩工具的开销可达数倍到数十倍，会扭曲时间分布，因此它给的是相对分布而非绝对耗时。

8　总结与延伸

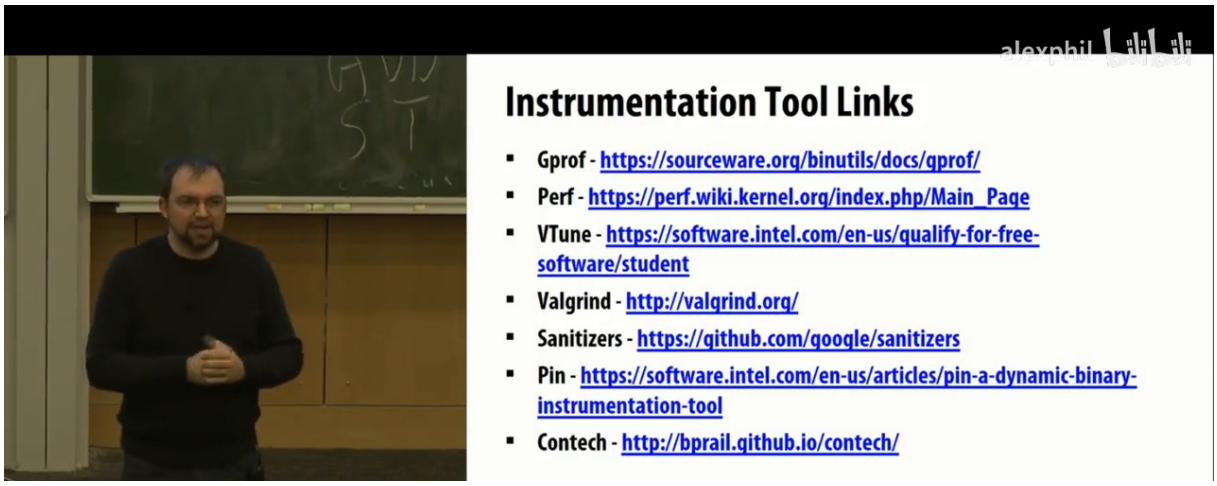


图 22：总结：各工具的资料链接与适用场景[22]

讲者在结尾给出各工具的资料链接，并把整讲收束。

[22] 视频画面时间区间：01:12:40–01:12:55。若为裁剪图，时间区间仍对应原视频画面。

把全讲串起来，这是一套由外向内逐层收窄的方法。第一层用 top 确认机器是否被他人占满、内存是否充足——注意 CPU 百分比按逻辑线程累加，八线程满载显示 800% 而非 100%。第二层用 gprof 做函数级计时，并用 Amdahl 定律判断哪一段值得优化；但必须结合评分规则排除无关的框架代码。第三层用 perf stat 读硬件计数器，关键是绝对数必须配总数构成比率，超过四个计数器会分时复用；读法是用排除法——IPC 0.4 偏低、分支预测 2.7% 正常、于是怀疑内存、缓存缺失率 24.5% 确认。第四层用 perf record 采样定位到行，但要记住微架构延迟会让位置偏移一到两条指令。最后的案例说明真正的瓶颈常在数据布局：双重指针数组的两级间接寻址带来了近十倍的减速，去掉后显著变快。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 这套方法是分层排除，不是一次性收集所有指标。每一层都用最便宜的工具排除一大类可能，只在必要时才进入下一层。颠倒顺序会导致在无关数据上花时间，或过早进入昂贵的插桩分析。
- 工具的结论必须与人对其业务语义的理解结合。gprof 显示构图占七成，但那段代码不参与评分，因此不是优化目标。这一步判断工具无法代劳。
- 采样位置偏移是架构特性而非工具缺陷。理解这一点，才能在看到” 不访存的行报缺失” 时继续排查而不是放弃工具。这也解释了为什么读报告需要同时看汇编上下文。
- 最有效的优化常来自数据布局，而布局问题不会在复杂度分析中显现。双重指针数组是典型：算法上等价，硬件上差近十倍。因此性能工程不能只停留在算法层面。

这堂课的边界在于它讲的是工具的使用方法与判读逻辑，不含各工具的完整参数。讲者多次说明示例来自 2016 年的作业，学生需要把方法迁移到当前作业。

两个可以直接从本讲数据提出的下一步问题：其一，插桩开销会改变时间分布，那么用插桩工具得到的相对占比，在多大程度上可以外推到未插桩的真实运行？其二，缓存缺失率 24.5% 已经很高，但若要进一步区分是容量缺失、冲突缺失还是伪共享，还需要什么工具或手段？